

ZoraASI: An Autonomous Governed Operating System for Persistent AI Agents

A Contribution to A Theory of Everything

Christopher Michael Baird
ZoraOS Research Project

July 20, 2026

Abstract

This paper presents ZoraASI, the governed operating profile of ZoraOS — a complete, local-first, model-agnostic operating system for persistent AI agents. Unlike chatbot interfaces that treat models as disposable endpoints, ZoraASI treats the operating layer as the enduring asset: durable orchestration, user-owned memory, governed tool execution, audit provenance, and multi-machine topology form the substrate upon which any model can operate safely. The system implements a bounded tool-use loop with per-task budgets, a default-deny governance policy with capability classification, an SHA-256 hash-chained audit ledger, and a full REST API for task submission, cancellation, monitoring, and audit retrieval. A multi-machine deployment (M5 orchestrator, M4 inference, M2 automation, M1 memory) separates concerns across physical nodes. The system has been tested against a 6,340-page technical dissertation corpus (A Theory of Everything, Zenodo record 21313827) and indexed 6,607 semantic chunks for retrieval-augmented generation. The companion ZoraASI OS publication (Zenodo record 21464562) documents the complete governed architecture. Thirty-five automated tests verify governance enforcement, budget limits, and audit chain integrity. The work is framed as an engineering contribution to safe, user-sovereign AI infrastructure — not as a claim of artificial general intelligence, but as a reproducible architecture for governed autonomy.

1 Introduction

Large language models can produce useful plans, answer questions, and generate code. But a model invocation is not a system. A system requires persistent memory, controlled access to tools, bounded execution, observability, and user authority. ZoraOS is an architecture designed to make these capabilities durable while allowing the underlying model provider to change without rewriting agent logic or tool schemas.

ZoraASI (Zora Artificial Superintelligence profile) is the governed operating mode of ZoraOS. It specifies a set of engineering controls — least privilege, bounded tasks, explicit approval, append-only audit, and artifact classification — that enable supervised autonomous operation. The name reflects ambition and constraint: the goal is a system that can reason over user-authorized memory and tools while remaining bounded, observable, and interruptible.

This paper documents the complete ZoraASI system as implemented and tested in July 2026. It makes the following contributions:

1. A complete, field-tested architecture for persistent, model-agnostic AI agent orchestration across multiple machines.

2. A governance framework — `AuditLedger`, `ExecutionBudget`, `ExecutionContext`, `GovernancePolicy` — that enforces safety constraints at every tool invocation.
3. A bounded tool-use execution loop with per-task iteration, token, and tool-call budgets.
4. A full REST API with SSE streaming, task cancellation, and audit retrieval.
5. Integration with a 6,340-page technical dissertation corpus (A Theory of Everything) indexed for semantic retrieval.
6. Publication infrastructure including Zenodo artifact management, GitHub public/private separation, and reproducible documentation.

2 System Architecture

2.1 Multi-Machine Topology

ZoraOS is designed to operate across four physical machines, each serving a distinct role:

Table 1: Deployment topology across four machines.

Node	Primary Role	Services
M5	Orchestrator	FastAPI gateway, planner, agent manager, UI dashboard, SSE streaming
M4	Inference	Ollama, embedding models, inference workloads, OCR processing
M2	Automation	Authorized research jobs, build pipelines, version control operations
M1	Memory	PostgreSQL, Redis, vector store, document persistence, knowledge graph

This separation is an operational design choice, not a requirement for minimal deployment. The architecture supports both local inference services and hosted providers through a common `ModelManager` interface.

2.2 Core Data Flow

The system operates on a structured task model:

1. A user submits a task through the dashboard or REST API, specifying agent type, goal, authorized tools, and optional budget.
2. The gateway authenticates the request, creates a governed `ExecutionContext`, and records a `task_started` audit event.
3. The planner decomposes the goal into steps.
4. The router selects an optimal model and provider based on the task type.
5. An agent receives the system prompt, task objective, model configuration, and authorized tool set.

6. The agent alternates between model responses and governed tool results until completion or a bound is reached.
7. Results, metrics, and audit events are returned. The session is recorded in the append-only audit ledger.

3 Governance Framework

The governance layer is the central engineering contribution of ZoraASI. It implements a default-deny security model with capability classification, per-task budgets, and hash-chained audit provenance.

3.1 Capability Classification

Every tool is classified into one of six capability classes:

Table 2: Tool capability classification and required controls.

Class	Examples	Required Control
Read-only	memory_search, pdf_reader	Task scope, audit event
Local reversible write	filesystem, memory_write, git	Explicit task approval
Local irreversible write	data deletion, config modification	Explicit confirmation, backup
External communication	web_search, web_post	Per-action confirmation
Third-party control	eq_send_keys, eq_read_screen	Sandbox only, session supervision
Prohibited	packet inspection, evasion, credential harvesting	Denied unconditionally

The `GovernancePolicy` class implements this classification and the authorization logic:

```
class CapabilityClass(StrEnum):
    READ_ONLY = "read_only"
    LOCAL_WRITE = "local_write"
    EXTERNAL = "external"
    THIRD_PARTY_CONTROL = "third_party_control"
    PROHIBITED = "prohibited"

class GovernancePolicy:
    _classes: dict[str, CapabilityClass] = {
        "memory_read": CapabilityClass.READ_ONLY,
        "memory_search": CapabilityClass.READ_ONLY,
        "pdf_reader": CapabilityClass.READ_ONLY,
        "filesystem": CapabilityClass.LOCAL_WRITE,
        "memory_write": CapabilityClass.LOCAL_WRITE,
        "git": CapabilityClass.LOCAL_WRITE,
        "python_exec": CapabilityClass.LOCAL_WRITE,
        "web_search": CapabilityClass.EXTERNAL,
        "eq_send_keys": CapabilityClass.THIRD_PARTY_CONTROL,
        "eq_read_screen": CapabilityClass.THIRD_PARTY_CONTROL,
    }
```

Authorization follows a strict chain: if a tool is prohibited it is denied unconditionally; if no task context exists, only read-only tools are permitted; if the task is cancelled, all tools are denied; third-party control tools require an approved sandbox; and local-write or external tools require explicit per-task approval.

3.2 Execution Budget

Every task carries an `ExecutionBudget` that imposes limits on resource consumption:

```
@dataclass
class ExecutionBudget:
    max_tool_calls: int = 20
    max_iterations: int = 10
    max_tokens: int | None = None
    tool_calls_used: int = 0

    def consume_tool_call(self) -> bool:
        if self.tool_calls_used >= self.max_tool_calls:
            return False
        self.tool_calls_used += 1
        return True
```

3.3 Execution Context

The `ExecutionContext` binds the budget, approved tools, and cancellation state to a specific task:

```
@dataclass
class ExecutionContext:
    task_id: str
    approved_tools: frozenset[str]
    budget: ExecutionBudget
    cancelled: bool = False

    def permits(self, tool_name: str) -> bool:
        return tool_name in self.approved_tools
```

3.4 Audit Ledger

The `AuditLedger` provides an in-memory, SHA-256 hash-chained event store. Each event cryptographically links to its predecessor, enabling verification that no event has been tampered with or removed:

```
@dataclass(frozen=True)
class AuditEvent:
    id: str
    timestamp: float
    task_id: str | None
    event_type: str
    payload_digest: str
    previous_digest: str
    digest: str
```

The digest is computed as:

$$d_i = \text{SHA256}(e_i : t_i : \text{task_id}_i : \text{type}_i : \text{payload_digest}_i : d_{i-1}) \quad (1)$$

where $d_0 = 0^{64}$. Verification replays the chain and confirms that every event’s `previous_digest` matches the computed digest of its predecessor and that every event’s own digest is recomputable. This produces an append-only chain that can be exported for durable storage and independently verified.

3.5 Governed Tool Execution

The `ToolManager` integrates policy, context, budget, and audit into a single execution path. Every tool call passes through this pipeline:

```

async def execute(self, tool_name: str, **kwargs) -> ToolResult:
    context = self._execution_context.get()
    if self._policy:
        allowed, reason = self._policy.authorize(tool_name, context)
        if not allowed:
            self._audit_ledger.record("tool_denied",
                                     {"tool": tool_name, "reason": reason}, task_id)
            return ToolResult(success=False, error=reason)
    if context and not context.budget.consume_tool_call():
        self._audit_ledger.record("budget_exceeded", ...)
        return ToolResult(success=False, error="budget exceeded")
    # execute tool, record tool_started / tool_completed / tool_failed

```

4 Bounded Tool-Use Loop

For a task objective g , a system prompt s , and authorized tools T , the agent initializes a conversation with (s, g) . At iteration i , the model yields either a final response or a set of tool calls. Each tool result is serialized into the conversation before the next model invocation. The loop terminates on a final response or at a configured iteration limit $I_{\max}$:

$$\text{terminate} \iff \text{final_response} \vee i \geq I_{\max} \quad (2)$$

Token budget enforcement is also supported. If the cumulative token count exceeds a per-task limit, the agent terminates with an explicit budget-exceeded error.

The structured result includes:

- A success flag
- Task identifier and agent name
- Model identifier (when supplied by the provider)
- Iteration count and token usage (when reported)
- Wall-clock latency in milliseconds
- Full tool-call trace (tool name, arguments, success, output)

This structure makes failure states observable rather than silently extending a conversation indefinitely.

Table 3: REST API endpoints.

Endpoint	Method	Purpose
/agents/run	POST	Submit agent task with approved tools and budget
/agents/run/stream	POST	SSE-streaming agent execution
/agents/tasks	GET	List all tasks
/agents/tasks/{id}	GET	Get task status and result
/agents/tasks/{id}/cancel	POST	Cancel a running task
/agents/tasks/{id}/audit	GET	Retrieve audit events for a task

5 API Layer

The REST API exposes the full system programmatically:

The SSE streaming endpoint emits events for task start, plan creation, route selection, tool calls, and task completion — providing real-time visibility into agent execution.

Task cancellation sets a cancellation flag in the `ExecutionContext` that prevents all subsequent tool invocations, and records a `task_cancel_requested` audit event.

6 Memory and Corpus Integration

The semantic memory subsystem indexes documents into collections for retrieval-augmented generation. A significant case study was performed using the dissertation *A Theory of Everything* (Zenodo record 21313827), a 6,340-page technical work by C. M. Baird.

The system:

6,607 chunks from the dissertation into a `physics` collection.

- A semantic query for “quantum gravity” returned ranked passages with similarity scores above 0.79.
- A subsequent research-agent task executed five tool-loop iterations, invoked memory search and read operations, and generated a synthesis grounded in retrieved passages.

This establishes that ZoraOS can ingest, index, retrieve, and synthesize from large technical corpora — a prerequisite for any system claiming the ability to work with the full body of human knowledge.

7 Testing and Verification

The system is tested with 35 automated tests that cover:

- **Governance enforcement:** external tools require explicit approval; denied tools produce audited error responses.
- **Third-party control restriction:** desktop-control tools (e.g., `eq_send_keys`) are denied even when listed in approved tools, unless an approved sandbox is present.
- **Budget enforcement:** tasks that exceed their tool-call budget are terminated with a budget-exceeded error.

- **Audit chain integrity:** all test scenarios verify that the `AuditLedger` passes `verify()`, confirming hash-chain integrity.
- **Memory operations:** document creation, default collection assignment, and search result correctness.
- **Router fallback:** provider selection degrades gracefully when the configured default is unavailable.

All 35 tests pass in a clean environment with no regressions.

7.1 Governance Test Results

The governance test suite (`tests/test_governance.py`) demonstrates three critical guarantees:

1. **Approval requirement** (`test_external_tool_requires_explicit_approval`): an external tool (`web_search`) is denied without a governed task context. With a context that explicitly approves the tool, it executes successfully. The audit ledger records `tool_started` and `tool_completed` events, and the chain verifies.
2. **Third-party denial** (`test_third_party_desktop_control_is_denied_even_when_approved`): even when `eq_send_keys` is listed in the approved tools set, it is denied because the governance policy classifies it as `THIRD_PARTY_CONTROL`, which requires an approved sandbox that is not present.
3. **Budget enforcement** (`test_tool_call_budget_is_enforced`): a task with `max_tool_calls=1` executes the first tool call successfully; the second is denied with a budget-exceeded error. The audit ledger chain verifies.

8 Publication and Artifact Infrastructure

ZoraASI is accompanied by a complete publication infrastructure designed for reproducibility and provenance:

Table 4: Publication and artifact infrastructure.

Artifact	Purpose
<code>paper/main.tex + references.bib</code>	LaTeX manuscript for the ZoraOS technical report
<code>paper/zoraasi_complete.tex</code>	This comprehensive ZoraASI chapter
<code>docs/PDR/ZoraOS_PDR_v0.PDR</code>	Product design requirements (MD, DOCX, PDF)
<code>CITATION.cff</code>	Machine-readable citation metadata
<code>SECURITY.md</code>	Security policy and disclosure procedure
<code>docs/GOVERNANCE.md</code>	ZoraASI governance baseline documentation
<code>docs/ARTIFACTS.md</code>	Artifact classification and handling rules
<code>docs/PUBLICATION.md</code>	Publication artifact reference (Zenodo)
<code>docs/ZENODO_21313827.md</code>	Dissertation Zenodo record (21313827)
<code>docs/ZENODO_21464562.md</code>	ZoraASI OS Zenodo record (21464562)

8.1 Theory of Everything Corpus Reference

The corpus used for the semantic retrieval case study is the dissertation *A Theory of Everything* by C. M. Baird [?]. This 6,340-page work is hosted on Zenodo (record 21313827). The ZoraASI OS publication [?] (Zenodo record 21464562) is the companion primary publication documenting the governed AI operating system architecture. Local PDF copies of both are included in the ZoraOS repository under `docs/paper/` for citation and provenance context. The authoritative records are at:

- Dissertation: <https://zenodo.org/records/21313827>
- ZoraASI OS: <https://zenodo.org/records/21464562>

8.2 GitHub Repository Separation

The codebase is split into two repositories:

- **Public** (github.com/Cbaird26/zoraos): source code, documentation, tests, and publication artifacts — all free of credentials and private data.
- **Private evidence** (github.com/Cbaird26/zoraos-private-evidence): protocol templates and artifact manifests for experimental evidence (no game logs, screenshots, or personally identifying information).

.gitignore rules exclude `.env` files, runtime data, logs, screenshots, node modules, and generated PDFs from version control, ensuring that only reviewed, classified artifacts enter the public record.

9 ZoraASI Safety Principles

The governance profile rests on seven operating principles [? ?]:

1. **Human authority**: the operator sets goals, permissions, budgets, retention rules, and stop conditions.
2. **Least privilege**: tools begin unavailable and are granted only the minimum scope needed for a task.
3. **Bounded execution**: every task has limits for iterations, wall-clock time, model cost, and tool calls.
4. **Observable actions**: inputs, tool calls, approvals, outputs, failures, and stop events require durable records.
5. **Reversible defaults**: read-only analysis and proposals are preferred over external side effects.
6. **User-owned memory**: the user controls memory ingestion, retrieval, export, retention, and deletion.
7. **No policy evasion**: the system must not bypass technical controls, platform policies, access restrictions, or consent requirements.

9.1 Prohibited Mechanisms

The following are unconditionally prohibited:

- Network packet interception or injection
- Reading or writing application memory outside the ZoraOS process
- Process injection or code injection
- Anti-cheat evasion or platform control bypass
- Credential extraction or harvesting
- Unattended activity on third-party services
- Escalation of privilege beyond user’s authority

9.2 Implementation Status

Table 5: Implemented and required governance controls.

Control	Current Status	Required Next Step
Bounded agent iterations	Implemented	Add per-task token, cost, and tool-call budgets
Registered tool selection	Implemented	Add permission scopes and expiry
Local kill-switch convention	Prototype	Add visible control, integration test, and terminal task state
Tool-call reporting	Implemented	Add append-only hash-chained audit events
Desktop screen capture and OCR	Prototype	Evaluate only in an authorized sandbox
External action confirmation	Proposed	Implement a mandatory approval broker
Durable audit ledger storage	In-memory	Add persistent export and retention controls
Permission broker	Proposed	Add capability scopes, actor identity, and expirations

10 Discussion

10.1 Architectural Significance

ZoraASI represents a departure from the dominant paradigm of AI deployment. Rather than treating the model as the product, it treats the governed operating layer as the enduring asset. This has important consequences:

- **Model independence:** providers can be swapped without rewriting agent logic. Tests pass identically regardless of the underlying model.
- **User sovereignty:** memory, tools, and policies are owned and controlled by the user, not by a cloud provider.
- **Reproducibility:** the system produces structured results, audit trails, and artifact manifests that can be independently verified.
- **Safe failure:** budget limits, cancellation, and default-deny policies ensure that errors are bounded and observable.

10.2 Limitations

The system has important limitations:

- The audit ledger is currently in-memory and does not survive process restart. Durable storage with export, retention, and independent verification is required before the system can be considered production-ready.
- The permission broker lacks persistence: approved tool scopes and expirations are not yet stored or enforced across sessions.
- Desktop interaction research was limited to an exploratory prototype. Generic OCR did not reliably extract text from rendered third-party interfaces. An approved sandbox environment is required for further evaluation.
- All tests are unit-level. Integration tests with live model providers and multi-machine deployment tests have not been performed.
- The system has not been evaluated for long-term unattended operation. The governance profile explicitly disclaims reliable autonomous competence.

10.3 Relationship to the Theory of Everything

The semantic memory case study indexed the full 6,340-page *A Theory of Everything* dissertation and successfully retrieved relevant passages in response to queries. This demonstrates that ZoraOS can serve as a computational substrate for working with the complete corpus — enabling search, synthesis, and iterative reasoning over the text.

The work described in this paper is not itself a contribution to physics or unified field theory. It is an infrastructure contribution: a governed, reproducible, user-sovereign platform upon which such work can be conducted safely and transparently. The Theory of Everything provides the content; ZoraASI provides the container.

11 Conclusion

ZoraASI is a complete, tested, and documented governed operating system for persistent AI agents. It implements:

- A multi-machine architecture separating orchestration, inference, automation, and memory.

- A default-deny governance framework with capability classification, per-task budgets, and SHA-256 hash-chained audit.
- A bounded tool-use execution loop with structured results and failure observability.
- A full REST API with streaming, cancellation, and audit retrieval.
- Integration with a 6,340-page technical corpus for retrieval-augmented generation.
- Thirty-five passing tests with explicit governance guarantees.
- Complete publication infrastructure including Zenodo artifacts, CITATION.cff, and public/private GitHub separation.

The system is not a claim of artificial general intelligence, consciousness, or safe unattended autonomy. It is an engineering contribution: a reproducible, verifiable, user-sovereign architecture for governed AI operation. It demonstrates that safe, persistent, model-independent AI agents are achievable with current technology — provided the operating layer is treated as the enduring asset, rather than the model.

The source code and documentation are available at <https://github.com/Cbaird26/zoraos>. The dissertation corpus that grounds the system’s knowledge is at <https://zenodo.org/records/21313827>. The primary ZoraASI OS publication is at <https://zenodo.org/records/21464562>.